Deployment of 3-Tier Applications Using AWS Services

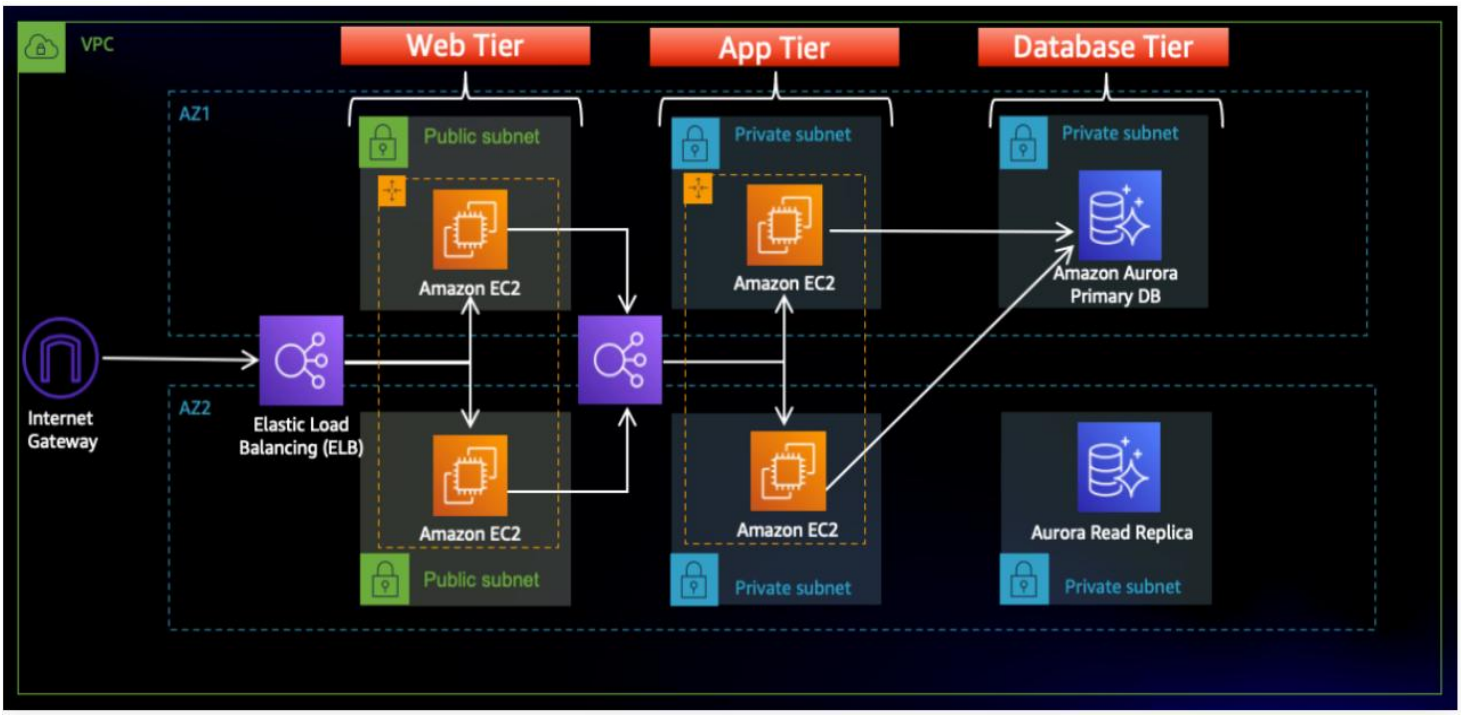
Name: Pooja Prakash Belgaumkar

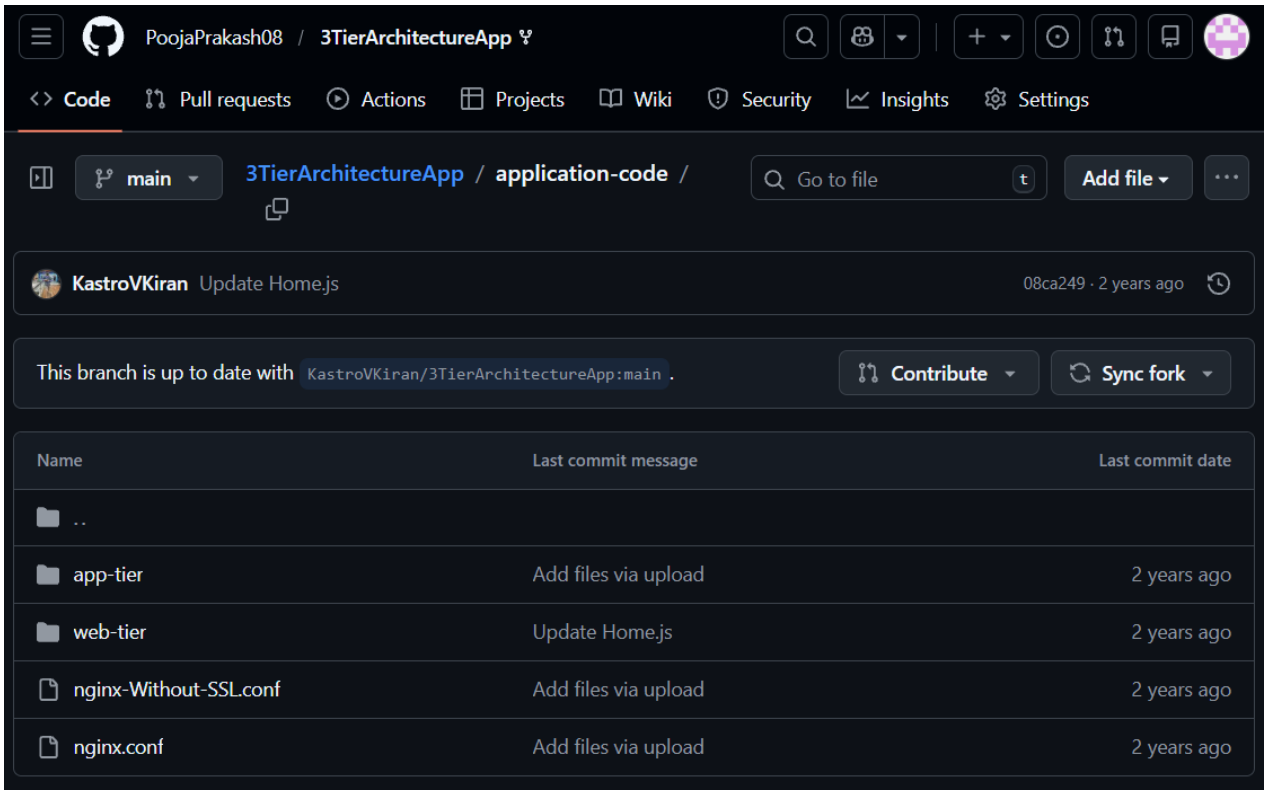
This project demonstrates the deployment of a secure, scalable, and highly available 3-Tier application architecture using AWS cloud services. The architecture is divided into three tiers:

- ✓ **Web Tier** – Provides the user interface and handles incoming user requests
- ✓ **Application Tier** – Contains backend logic and processes requests from the web tier
- ✓ **Database Tier** – Stores application data securely

To ensure high availability and fault tolerance, the infrastructure is deployed across two Availability Zones within a single VPC. Public and private subnets are used to isolate resources and enhance security.

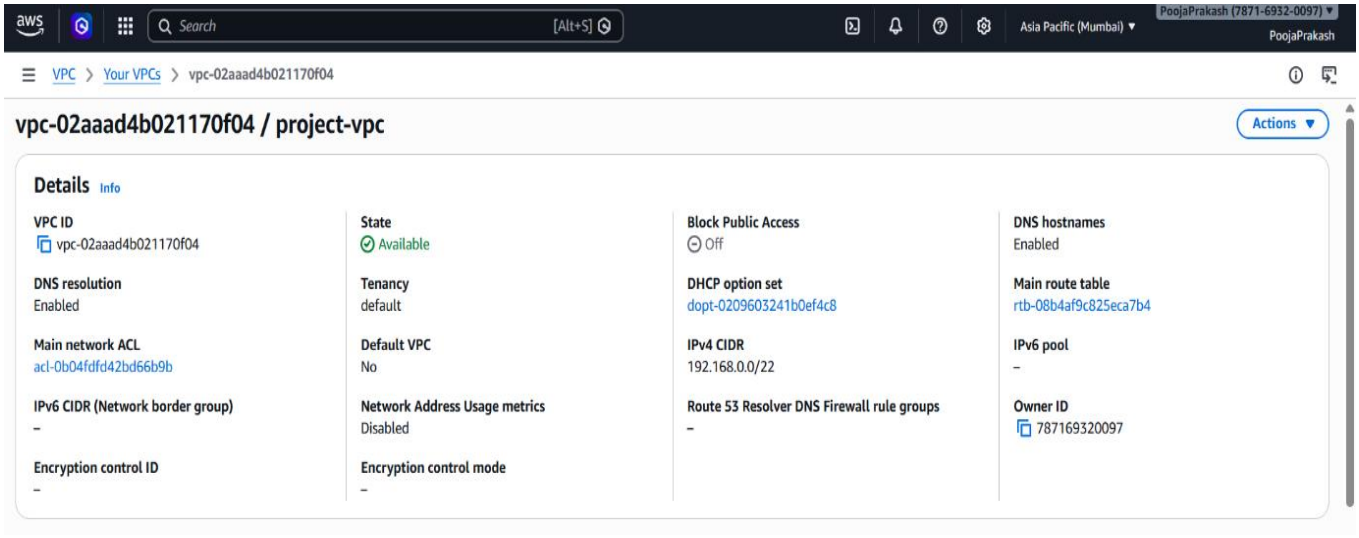
- ➢ Web tier instances are placed behind an **External Application Load Balancer** to allow access from the internet
- ➢ Application tier instances are placed behind an **Internal Load Balancer** to restrict access within the VPC
- ➢ Database tier is deployed in **private subnets** to prevent direct public access
- ➢ All resources are deployed in the same AWS region to ensure proper connectivity and performance

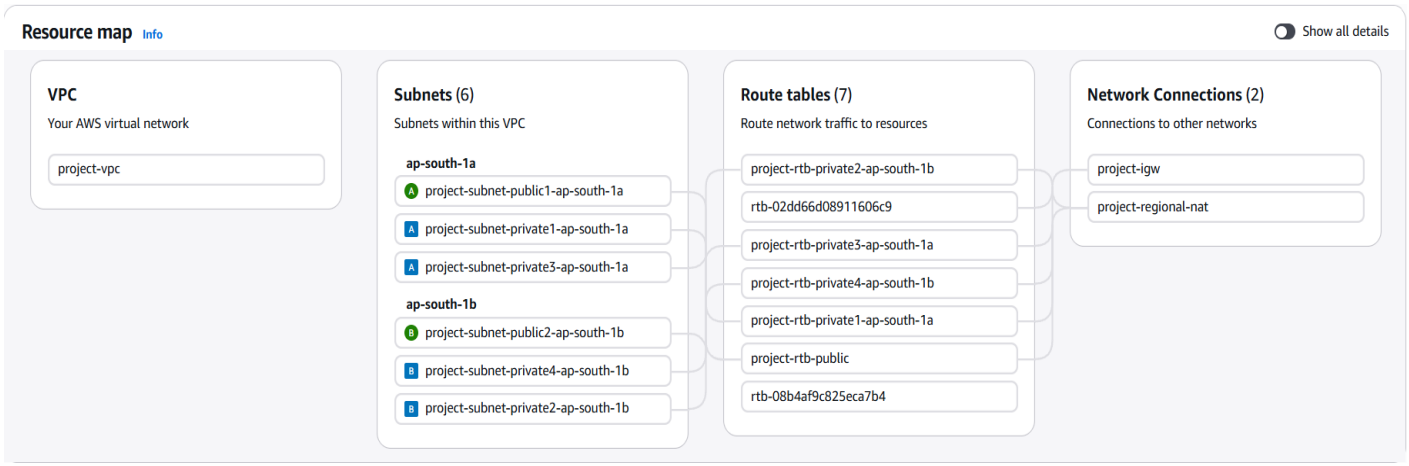

VPC
Web Tier
App Tier
Database Tier
AZ1
Public subnet
Private subnet
Private subnet
Amazon EC2
Amazon EC2
Amazon Aurora
Primary DB
Internet
Gateway
Elastic Load
Balancing (ELB)
AZ2
Amazon EC2
Amazon EC2
Aurora Read Replica
Public subnet
Private subnet
Private subnet


PoojaPrakash08 / 3TierArchitectureApp
Code
Pull requests
Actions
Projects
Wiki
Security
Insights
Settings
main
3TierArchitectureApp / application-code /
Go to file
t
Add file
KastroVKiran Update Home.js
08ca249 · 2 years ago
This branch is up to date with KastroVKiran/3TierArchitectureApp:main .
Contribute
Sync fork
Name
Last commit message
Last commit date
..
app-tier
Add files via upload
2 years ago
web-tier
Update Home.js
2 years ago
nginx-Without-SSL.conf
Add files via upload
2 years ago
nginx.conf
Add files via upload
2 years ago

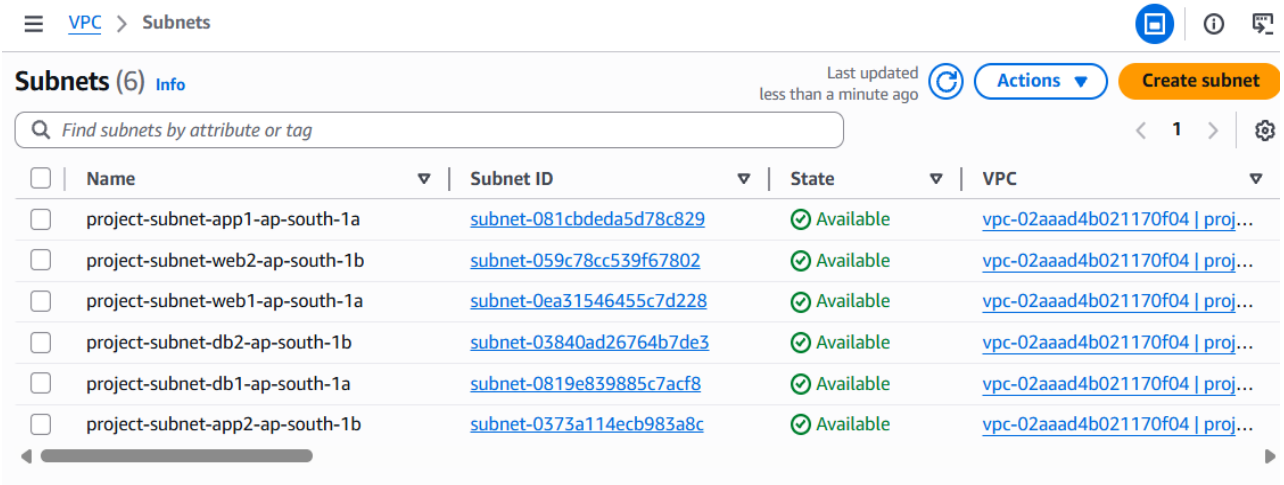
1. VPC Creation

- Create the VPC named **project-vpc** with 2 public subnets (for Web Tier and External Load Balancer) and 4 private subnets (2 for App Tier, 2 for Database Tier)





- Rename the subnets properly for easy identification.



- Create the following security groups:
  a. Web-ALB-SG → for External Load Balancer
     - ✓ Purpose: Allows users to access the application
     - ✓ Inbound Rules:
       i. HTTP – Port 80 – Source: 0.0.0.0/0

  b. Web-SG → for Web Tier Instance
     - ✓ Purpose: Allows traffic from External Load Balancer
     - ✓ Inbound Rules:
       i. HTTP – Port 80 – Source: Web-ALB-SG
       ii. SSH (optional) – Source: VPC

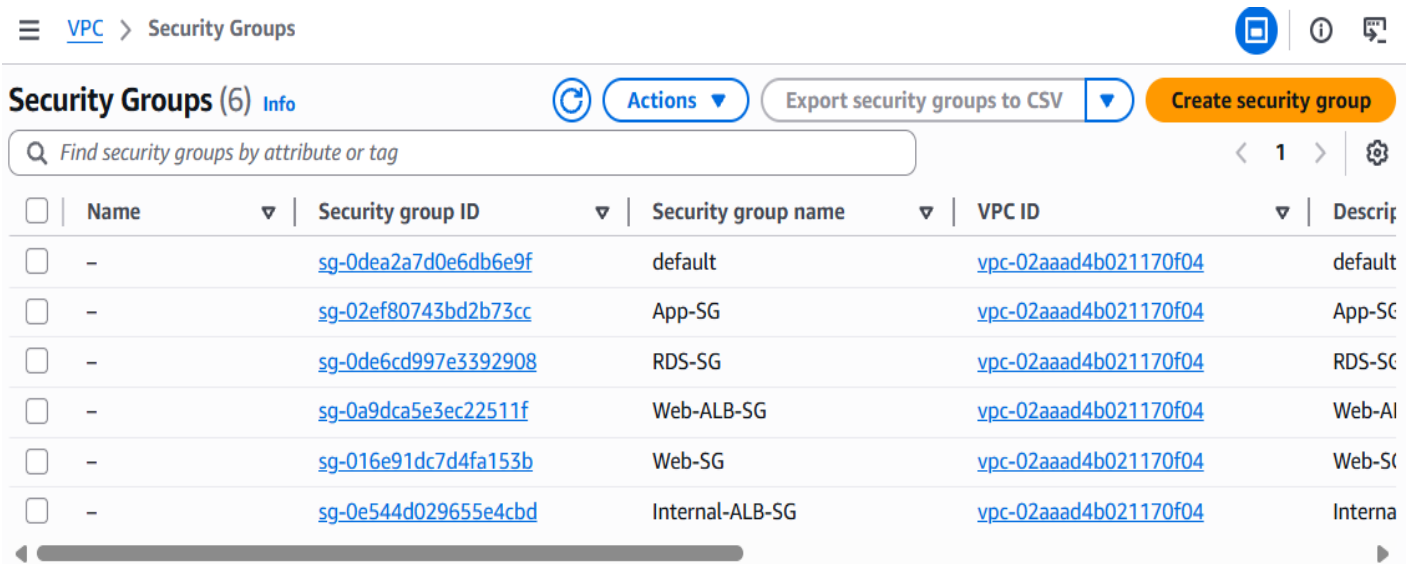
  c. App-SG → for Application Tier Instance
     - ✓ Purpose: Allows traffic only from Web Tier and Internal Load Balancer
     - ✓ Inbound Rules:
       i. Port 4000 – Source: Internal-ALB-SG
       ii. SSH via SSM only

d. Internal-ALB-SG →

- ✓ Purpose: Allows communication inside VPC
- ✓ Inbound Rules:
  - i. Port 4000 – Source: Web-SG

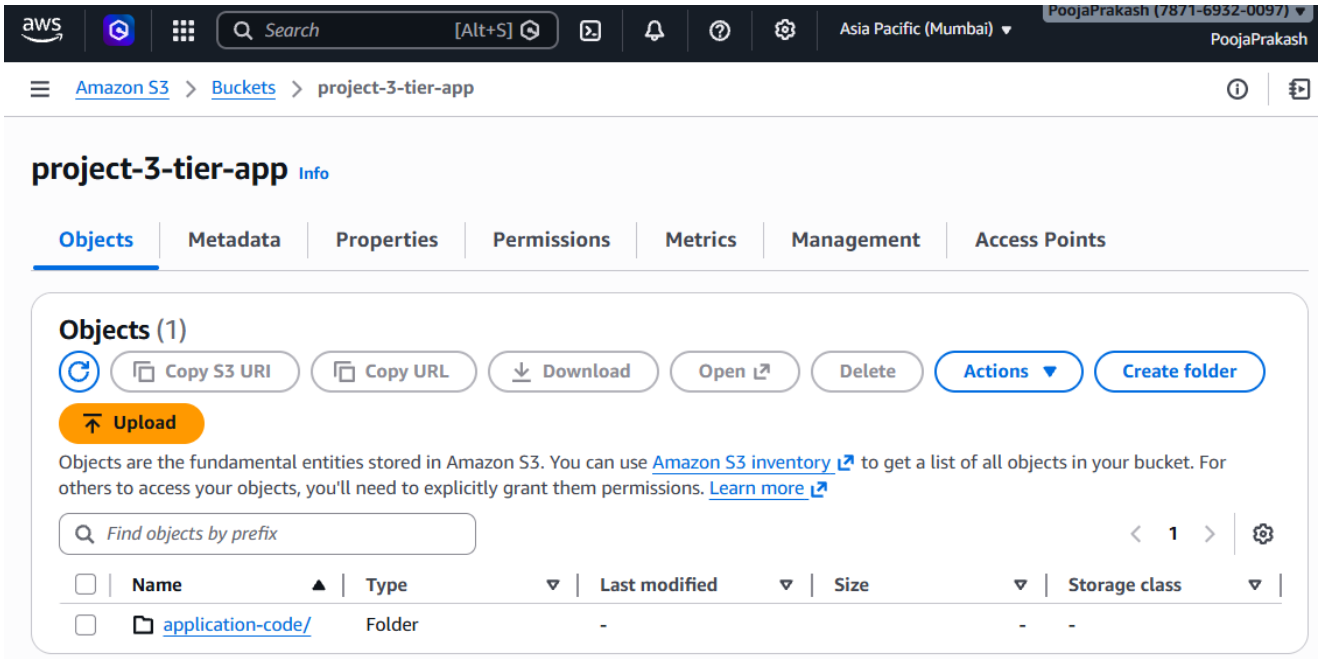
e. RDS-SG → for Database Tier

- ✓ Purpose: Allows database access only from Application Tier
- ✓ Inbound Rules:
  - i. MySQL – Port 3306 – Source: App-SG

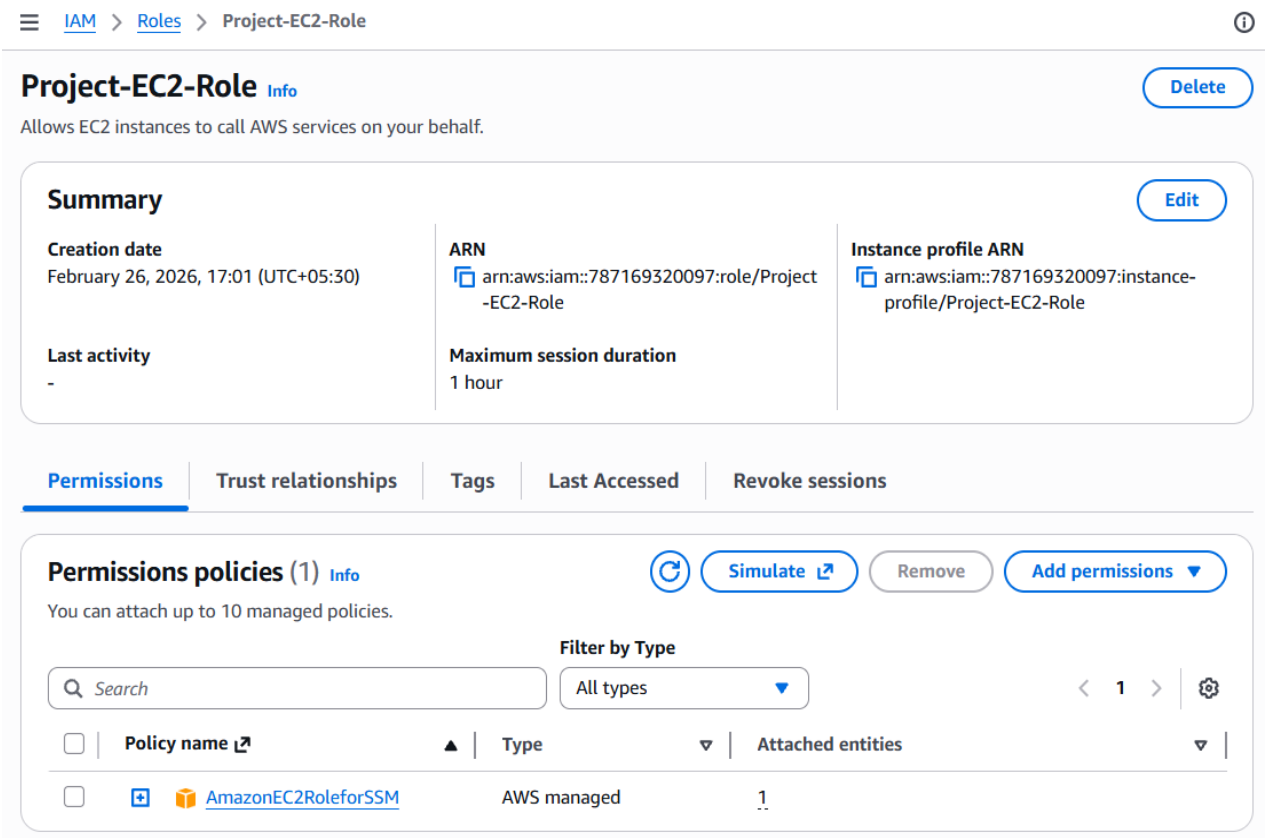


2. S3 Bucket and IAM Role Setup

- Directly uploading the source code into instance may hassle for further process thus we will create the s3 bucket with named **project-3-tier-app** in the same region and upload the application source in this bucket
- This allows EC2 instances to securely download application code.

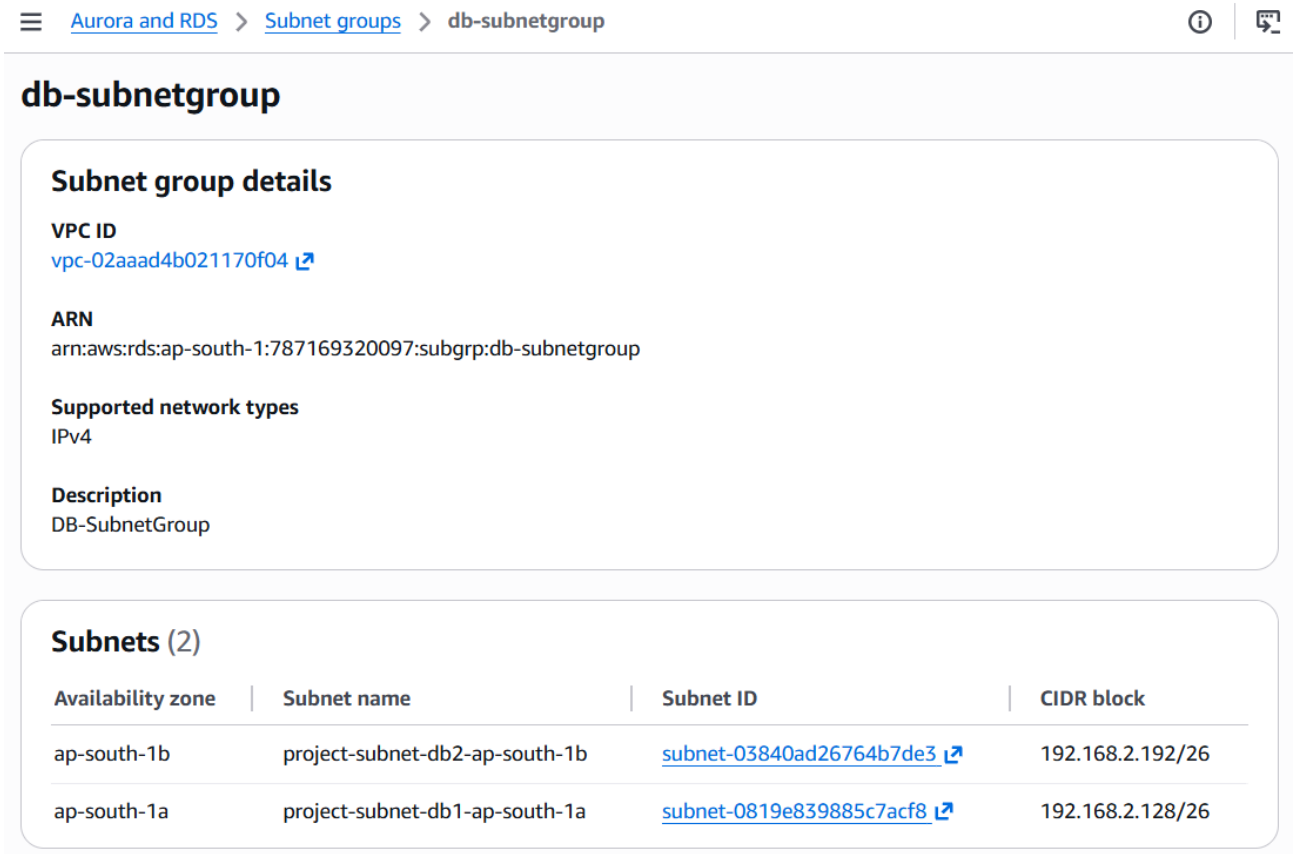


- Create IAM Role with policy AmazonEC2RoleforSSM permission and Attach this IAM role to EC2 instances.
- It allows EC2 to access S3 securely and Enables Session Manager (SSM) to connect to private instances without SSH.

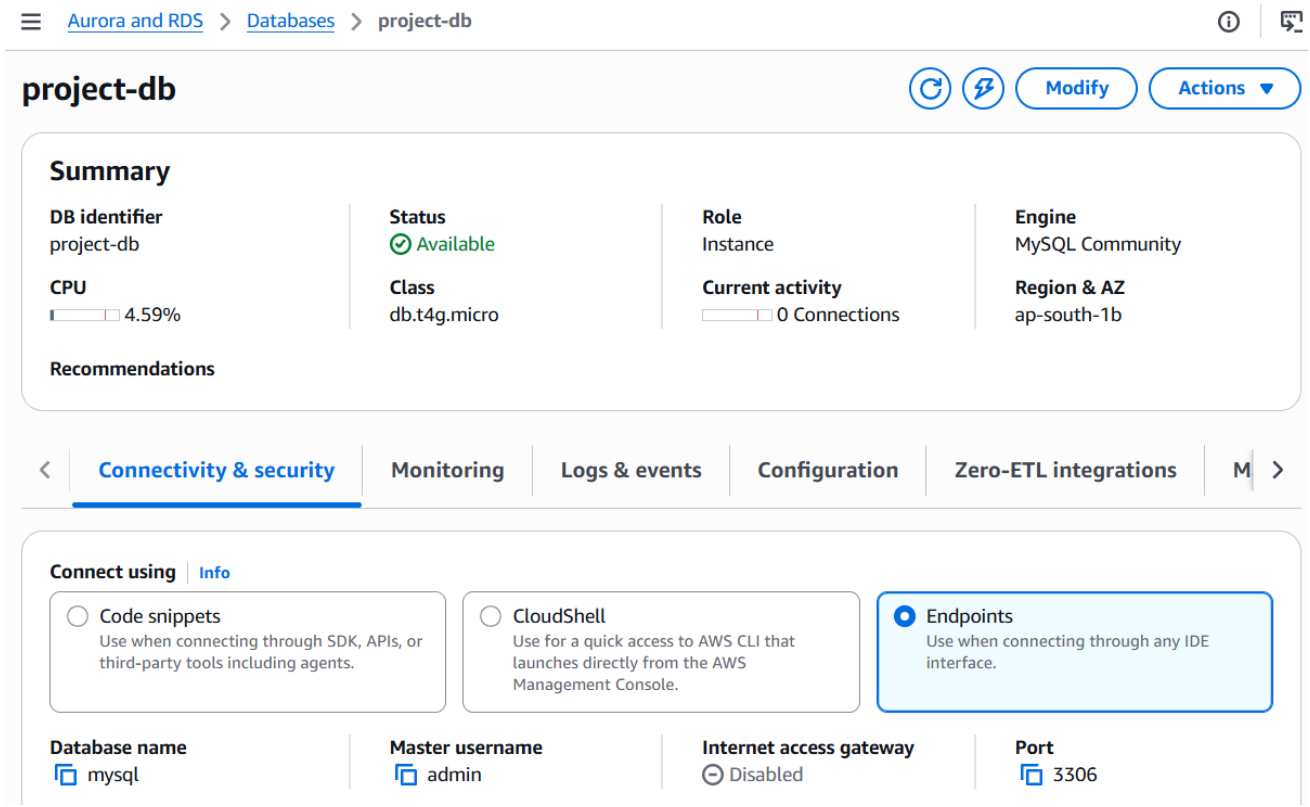


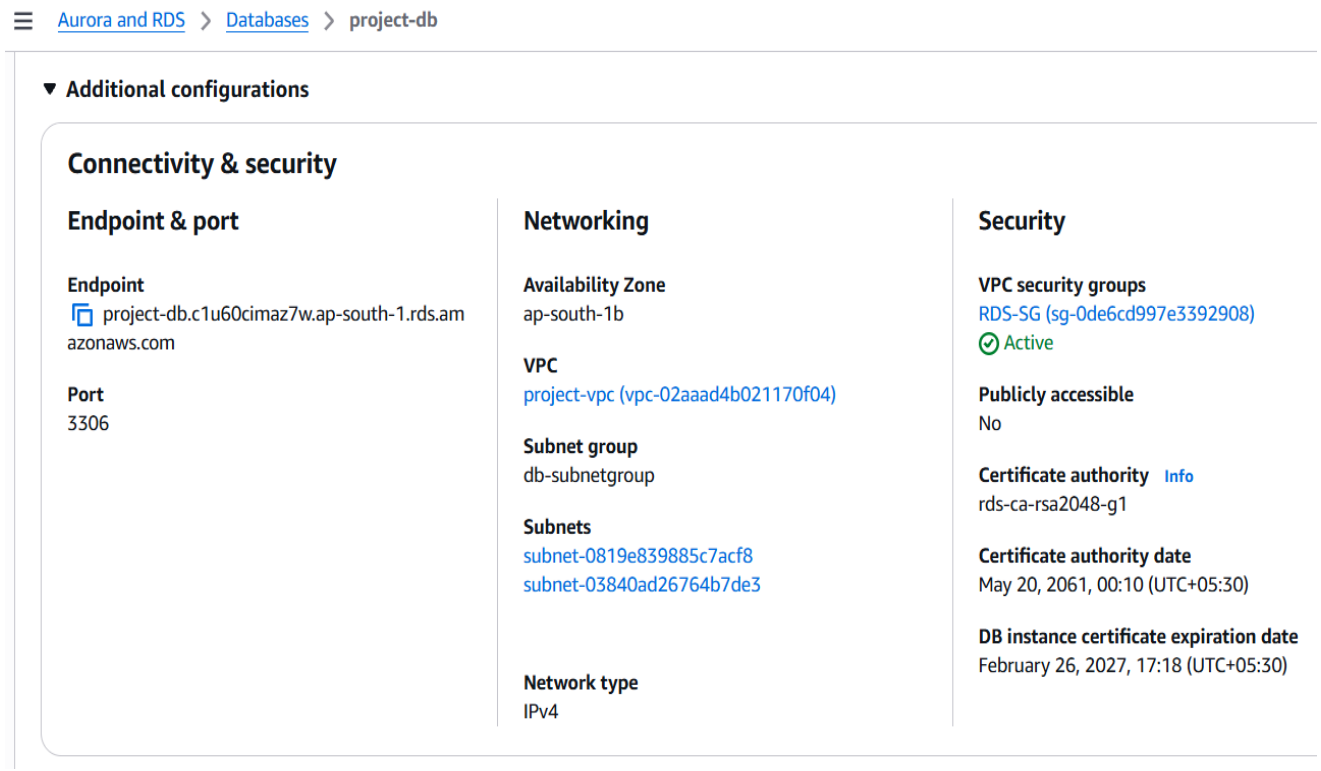
3. Database Configuration

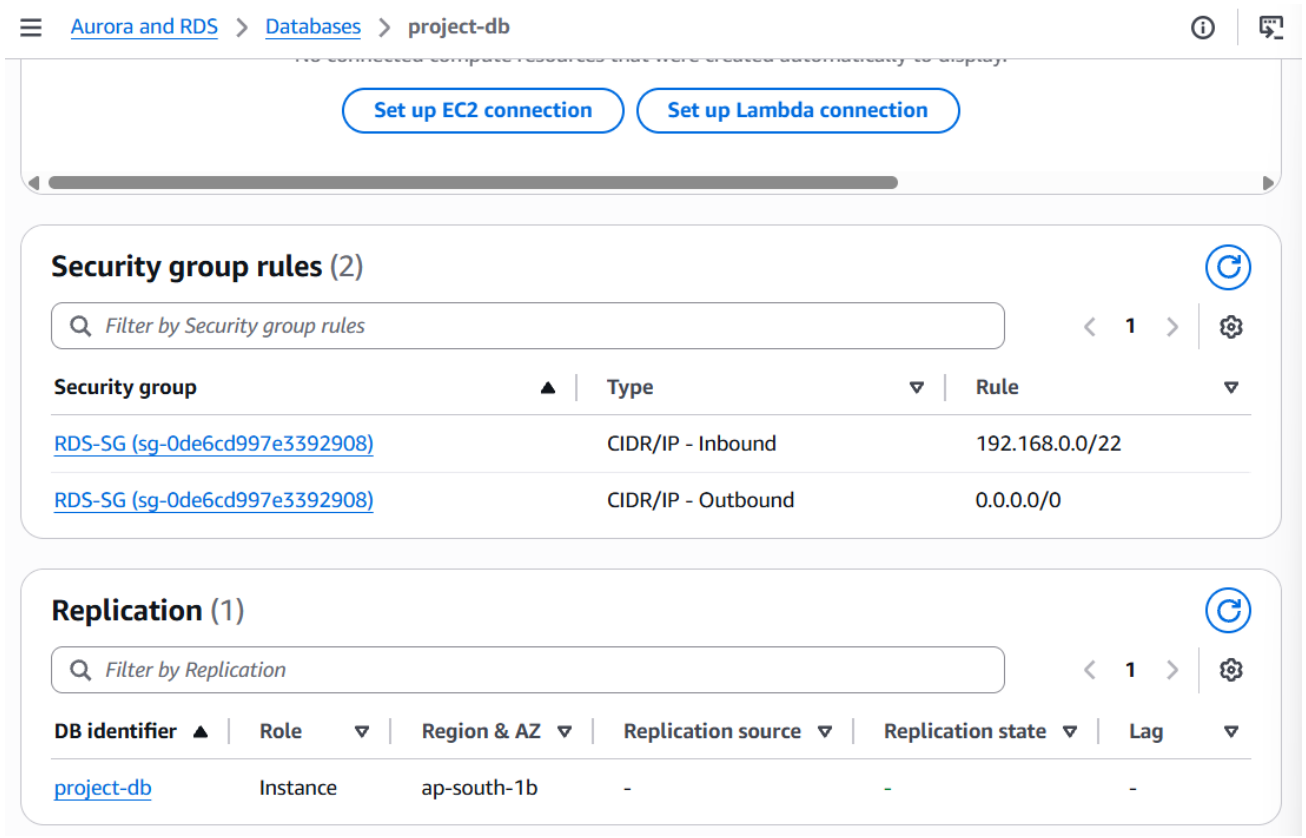
- Now we will create subnet group using only Private DB subnets to ensures database is accessible only from private network.



- Then create the RDS Database with configuration:
    - Engine: MySQL
    - Deployment: Private subnets
    - Security Group: RDS-SG
    - Public access: Disabled
- It provides secure and managed database service.


Aurora and RDS > Databases > project-db
project-db
Modify
Actions
Summary
DB identifier
project-db
Status
Available
Role
Instance
Engine
MySQL Community
CPU
4.59%
Class
db.t4g.micro
Current activity
0 Connections
Region & AZ
ap-south-1b
Recommendations
Connectivity & security
Monitoring
Logs & events
Configuration
Zero-ETL integrations
Connect using
Info
Code snippets
Use when connecting through SDK, APIs, or third-party tools including agents.
CloudShell
Use for a quick access to AWS CLI that launches directly from the AWS Management Console.
Endpoints
Use when connecting through any IDE interface.
Database name
mysql
Master username
admin
Internet access gateway
Disabled
Port
3306


Aurora and RDS > Databases > project-db
Additional configurations
Connectivity & security
Endpoint & port
Endpoint
project-db.c1u60cimaz7w.ap-south-1.rds.amazonaws.com
Port
3306
Networking
Availability Zone
ap-south-1b
VPC
project-vpc (vpc-02aaad4b021170f04)
Subnet group
db-subnetgroup
Subnets
subnet-0819e839885c7acf8
subnet-03840ad26764b7de3
Network type
IPv4
Security
VPC security groups
RDS-SG (sg-0de6cd997e3392908)
Active
Publicly accessible
No
Certificate authority
Info
rds-ca-rsa2048-g1
Certificate authority date
May 20, 2061, 00:10 (UTC+05:30)
DB instance certificate expiration date
February 26, 2027, 17:18 (UTC+05:30)



4. Application Tier Setup

- Create App Tier EC2 Instance named **App-Tier-Instance** inside the app tier subnet and connect it to instance through SSM session management as the subnet is private

EC2 > Instances > i-07640aa6bce4d8834

Instance summary for i-07640aa6bce4d8834 (App-Tier-Instance) Info

Connect | Instance state | Actions

Updated less than a minute ago

Instance ID i-07640aa6bce4d8834	**Public IPv4 address** –	**Private IPv4 addresses** 192.168.2.22
IPv6 address –	**Instance state** Running	**Public DNS** –
Hostname type IP name: ip-192-168-2-22.ap-south-1.compute.internal	**Private IP DNS name (IPv4 only)** ip-192-168-2-22.ap-south-1.compute.internal	
Answer private resource DNS name –	**Instance type** t3.micro	**Elastic IP addresses** –
Auto-assigned IP address –	**VPC ID** vpc-02aaad4b021170f04 (project-vpc)	**AWS Compute Optimizer finding** Opt-in to AWS Compute Optimizer for recommendations. | Learn more
IAM role Project-EC2-Role	**Subnet ID** subnet-081cbdeda5d78c829 (project-subnet-app1-ap-south-1a)	**Auto Scaling Group name** –

EC2 > Instances > i-07640aa6bce4d8834 > Connect to instance

Connect Info

Connect to an instance using the browser-based client.

Instance ID i-07640aa6bce4d8834 (App-Tier-Instance)	**VPC ID** vpc-02aaad4b021170f04	**Security groups** sg-02ef80743bd2b73cc (App-SG)	**IAM role** Project-EC2-Role

EC2 Instance Connect | **SSM Session Manager** | SSH client | EC2 serial console

SSM agent status

Ping status Online	**Last ping time** Thu Feb 26 2026 17:43:32 GMT+0530 (India Standard Time)	**Session Manager connection status** Connected	**Agent version** 3.3.3598.0

Session Manager usage:

- Connect to your instance without SSH keys, a bastion host, or opening any inbound ports.
- Sessions are secured using an AWS Key Management Service key.
- You can log session commands and details in an Amazon S3 bucket or CloudWatch Logs log group.
- Configure sessions on the Session Manager Preferences page.

Cancel | Connect

- Inside the App-Tier-Instance ec2-user we will install the MySQL to communicate with the database
- Connect to the database using the endpoint of Database and perform basic configuration

```
[root@ip-192-168-2-22 ec2-user]# mysql --version
mysql  Ver 15.1 Distrib 10.5.29-MariaDB, for Linux (x86_64) using  EditLine wrapper
[root@ip-192-168-2-22 ec2-user]# mysql -h project-db.c1u60cimaz7w.ap-south-1.rds.amazonaws.com -u adm
in -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MySQL connection id is 46
Server version: 8.0.40 Source distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MySQL [(none)]>
```

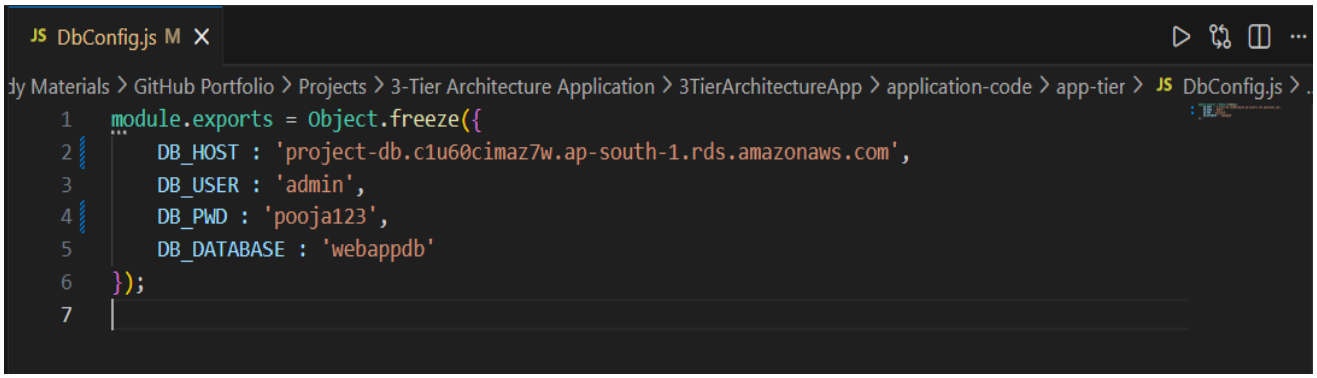
- Create the **webapp** Database and inside the Database create table add sample data.

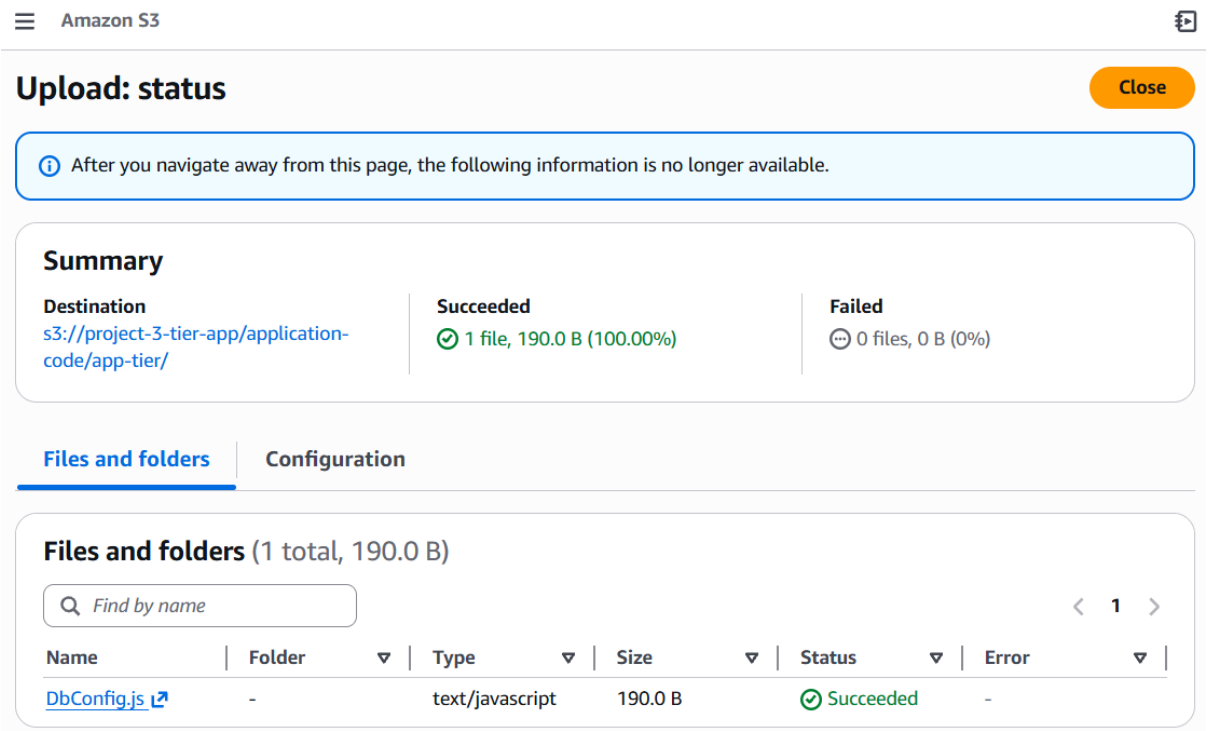
```
MySQL [(none)]> CREATE DATABASE webappdb;
Query OK, 1 row affected (0.022 sec)

MySQL [(none)]> SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| sys                |
| webappdb           |
+--------------------+
5 rows in set (0.011 sec)

MySQL [(none)]> USE webappdb;
Database changed
MySQL [webappdb]>
```

```
MySQL [webappdb]> CREATE TABLE IF NOT EXISTS transactions(
    ->   id INT NOT NULL AUTO_INCREMENT,
    ->   amount DECIMAL(10,2),
    ->   description VARCHAR(100),
    ->   PRIMARY KEY(id)
    -> );
Query OK, 0 rows affected (0.064 sec)

MySQL [webappdb]> SHOW TABLES;
+--------------------+
| Tables_in_webappdb |
+--------------------+
| transactions       |
+--------------------+
1 row in set (0.002 sec)

MySQL [webappdb]>
```

```
MySQL [webappdb]> INSERT INTO transactions (amount, description) VALUES ('400', 'groceries');
Query OK, 1 row affected (0.006 sec)

MySQL [webappdb]> SELECT * FROM transactions;
+----+--------+-------------+
| id | amount | description |
+----+--------+-------------+
|  1 | 400.00 | groceries   |
+----+--------+-------------+
1 row in set (0.002 sec)

MySQL [webappdb]> exit
Bye
[root@ip-192-168-2-22 ec2-user]#
```

- Update the Application Configuration to with our created database credentials and replace the file in the s3 bucket with new updated file





- Install Node.js and PM2 process manager
  - Purpose: Node.js runs application and PM2 keeps application running continuously

```
[root@ip-192-168-2-22 ec2-user]# curl -o- https://raw.githubusercontent.com/avizway1/aws_3tier_archite
cture/main/install.sh | bash
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100 14926 100 14926   0     0 46952     0  --:--:-- --:--:-- --:--:-- 46937
=> Downloading nvm as script to '/root/.nvm'

=> Appending nvm source string to /root/.bashrc
=> Appending bash_completion source string to /root/.bashrc
=> Close and reopen your terminal to start using nvm or run the following to use it now:

export NVM_DIR="$HOME/.nvm"
[ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh"  # This loads nvm
[ -s "$NVM_DIR/bash_completion" ] && \. "$NVM_DIR/bash_completion"  # This loads nvm bash_completion
[root@ip-192-168-2-22 ec2-user]# source ~/.bashrc
[root@ip-192-168-2-22 ec2-user]# nvm install 16
Downloading and installing node v16.20.2...
Downloading https://nodejs.org/dist/v16.20.2/node-v16.20.2-linux-x64.tar.xz...
######################################################################################################## 100.0%
Computing checksum with sha256sum
Checksums matched!
Now using node v16.20.2 (npm v8.19.4)
Creating default alias: default -> 16 (-> v16.20.2)
[root@ip-192-168-2-22 ec2-user]# nvm use 16
Now using node v16.20.2 (npm v8.19.4)
[root@ip-192-168-2-22 ec2-user]#
```

```
[root@ip-192-168-2-22 ec2-user]# npm install -g pm2

added 133 packages, and audited 134 packages in 10s

13 packages are looking for funding
  run `npm fund` for details

1 low severity vulnerability

Some issues need review, and may require choosing
a different dependency.

Run `npm audit` for details.
npm notice
npm notice New major version of npm available! 8.19.4 -> 11.11.0
npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.11.0
npm notice Run npm install -g npm@11.11.0 to update!
npm notice
[root@ip-192-168-2-22 ec2-user]#
```

- Download application code from S3 bucket and start the application.

```
[root@ip-192-168-2-22 ~]# pwd
/root
[root@ip-192-168-2-22 ~]# sudo aws s3 cp s3://project-3-tier-app/application-code/app-tier/ app-tier --recursive
download: s3://project-3-tier-app/application-code/app-tier/DbConfig.js to app-tier/DbConfig.js
download: s3://project-3-tier-app/application-code/app-tier/README.md to app-tier/README.md
download: s3://project-3-tier-app/application-code/app-tier/TransactionService.js to app-tier/TransactionService.js
download: s3://project-3-tier-app/application-code/app-tier/package-lock.json to app-tier/package-lock.json
download: s3://project-3-tier-app/application-code/app-tier/package.json to app-tier/package.json
download: s3://project-3-tier-app/application-code/app-tier/index.js to app-tier/index.js
[root@ip-192-168-2-22 ~]# ls
app-tier
```

- Inside the app-tier folder we have inde.js file we have to start this file using pm2 start index.js command then we will see status as online.

```
[root@ip-192-168-2-22 ~]# cd app-tier
[root@ip-192-168-2-22 app-tier]# npm install

added 68 packages, and audited 69 packages in 2s

2 packages are looking for funding
  run `npm fund` for details

7 vulnerabilities (3 low, 4 high)

To address all issues, run:
  npm audit fix

Run `npm audit` for details.
[root@ip-192-168-2-22 app-tier]# ls
DbConfig.js  TransactionService.js  node_modules       package.json
README.md    index.js               package-lock.json
[root@ip-192-168-2-22 app-tier]#
```

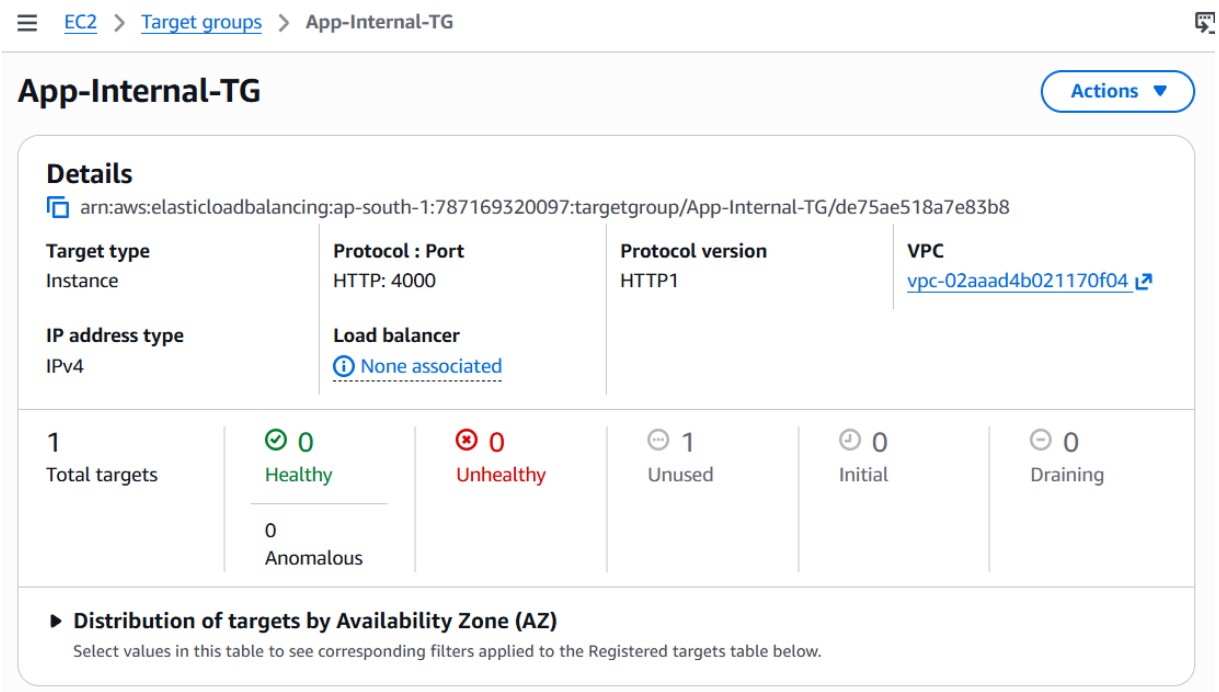
```
[root@ip-192-168-2-22 app-tier]# ls
DbConfig.js  TransactionService.js  node_modules       package.json
README.md    index.js               package-lock.json
[root@ip-192-168-2-22 app-tier]# pm2 start index.js

                      -------------

__/\\\\\\\\\\\\\____/\\\\____________/\\\\____/\\\\\\\\\_____
 _\/\\\/////////\\\_\/\\\\\\________/\\\\\\__/\\\///////\\\___
  _\/\\\_______\/\\\_\/\\\//\\\____/\\\//\\\_\///______\//\\\__
   _\/\\\\\\\\\\\\\/__\/\\\\///\\\/\\\/_\/\\\___________/\\\/___
    _\/\\\/////////____\/\\\__\///\\\/___\/\\\________/\\\//_____
     _\/\\\_____________\/\\\____\///_____\/\\\_____/\\\//________
      _\/\\\_____________\/\\\_____________\/\\\___/\\\/___________
       _\/\\\_____________\/\\\_____________\/\\\__/\\\\\\\\\\\\\\\_
        _\///______________\///______________\///__\///////////////__
```

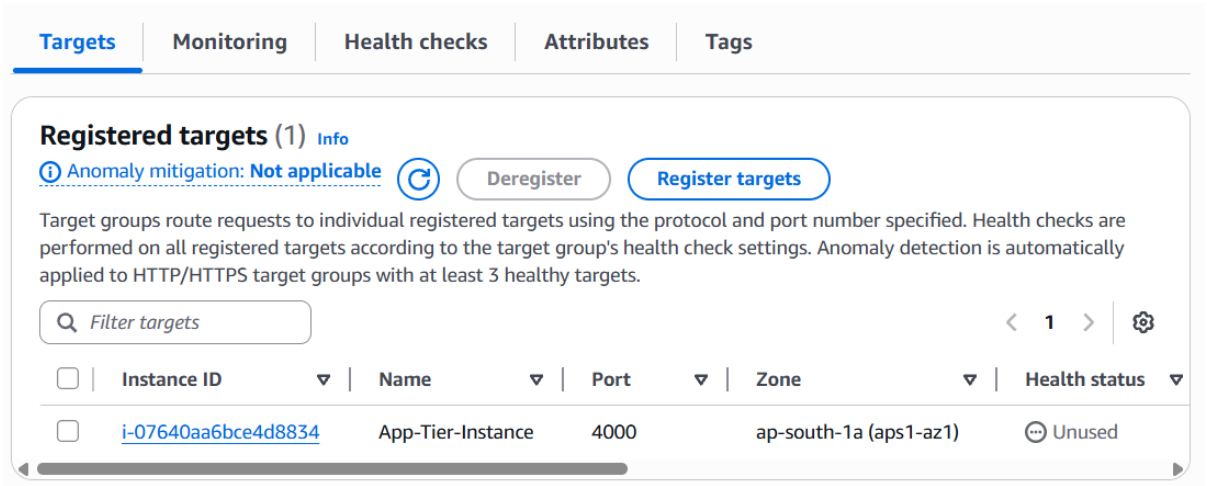
```
[root@ip-192-168-2-22 app-tier]# pm2 status
┌────┬──────────────────┬──────────┬────┬──────────┬──────────┬──────────┐
│ id │ name             │ mode     │ ↺  │ status   │ cpu      │ memory   │
├────┼──────────────────┼──────────┼────┼──────────┼──────────┼──────────┤
│ 0  │ index            │ fork     │ 0  │ online   │ 0%       │ 51.9mb   │
└────┴──────────────────┴──────────┴────┴──────────┴──────────┴──────────┘
[root@ip-192-168-2-22 app-tier]# pm2 logs
[TAILING] Tailing last 15 lines for [all] processes (change the value with --lines option)
/root/.pm2/pm2.log last 15 lines:
PM2        | 2026-02-26T12:54:52: PM2 log: Node.js version      : 16.20.2
PM2        | 2026-02-26T12:54:52: PM2 log: Current arch         : x64
PM2        | 2026-02-26T12:54:52: PM2 log: PM2 home             : /root/.pm2
PM2        | 2026-02-26T12:54:52: PM2 log: PM2 PID file         : /root/.pm2/pm2.pid
PM2        | 2026-02-26T12:54:52: PM2 log: RPC socket file      : /root/.pm2/rpc.sock
PM2        | 2026-02-26T12:54:52: PM2 log: BUS socket file      : /root/.pm2/pub.sock
PM2        | 2026-02-26T12:54:52: PM2 log: Application log path : /root/.pm2/logs
PM2        | 2026-02-26T12:54:52: PM2 log: Worker Interval      : 30000
PM2        | 2026-02-26T12:54:52: PM2 log: Process dump file    : /root/.pm2/dump.pm2
PM2        | 2026-02-26T12:54:52: PM2 log: Concurrent actions   : 2
PM2        | 2026-02-26T12:54:52: PM2 log: SIGTERM timeout      : 1600
PM2        | 2026-02-26T12:54:52: PM2 log: Runtime Binary       : /root/.nvm/versions/node/v16.20.2/bi
n/node
PM2        | 2026-02-26T12:54:52: PM2 log: ================================================================
====================
PM2        | 2026-02-26T12:54:52: PM2 log: App [index:0] starting in -fork mode-
PM2        | 2026-02-26T12:54:52: PM2 log: App [index:0] online

/root/.pm2/logs/index-error.log last 15 lines:
/root/.pm2/logs/index-out.log last 15 lines:
0|index    | AB3 backend app listening at http://localhost:4000
```

- Start the application using pm2 commands and verify
  curl http://localhost:4000/health the expected output should be “This is the health check”

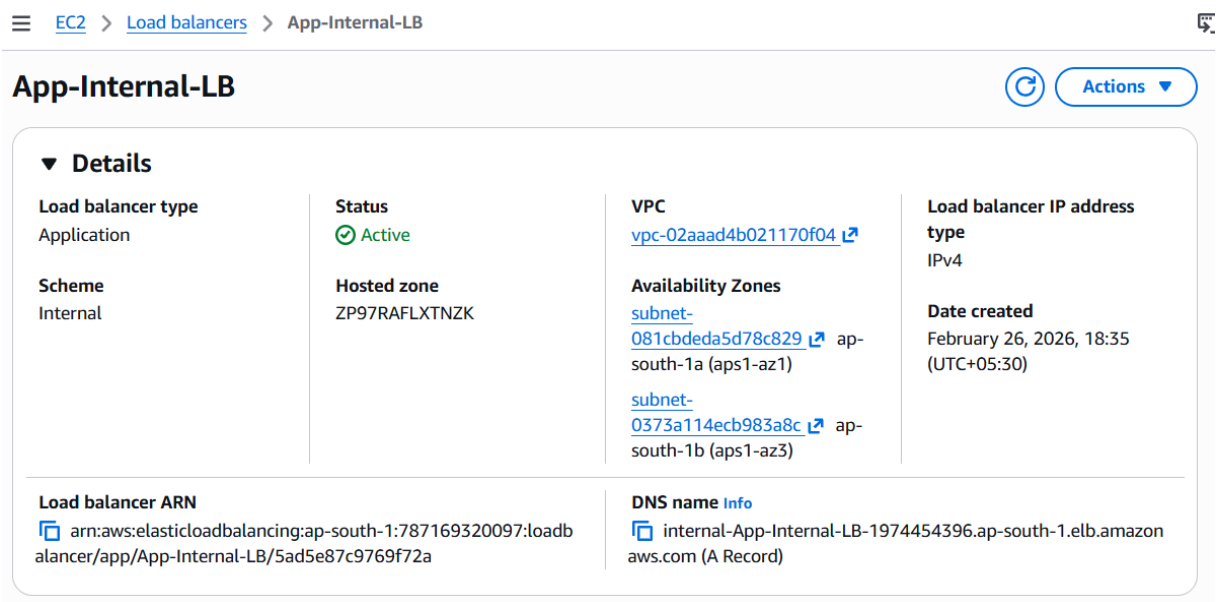
```
[PM2] [v] Command successfully executed.
+---------------------------------------+
[PM2] Freeze a process list on reboot via:
$ pm2 save

[PM2] Remove init script via:
$ pm2 unstartup systemd
[root@ip-192-168-2-22 app-tier]# pm2 save
[PM2] Saving current process list...
[PM2] Successfully saved in /root/.pm2/dump.pm2
[root@ip-192-168-2-22 app-tier]# curl http://localhost:4000/health
"This is the health check"[root@ip-192-168-2-22 app-tier]#
```

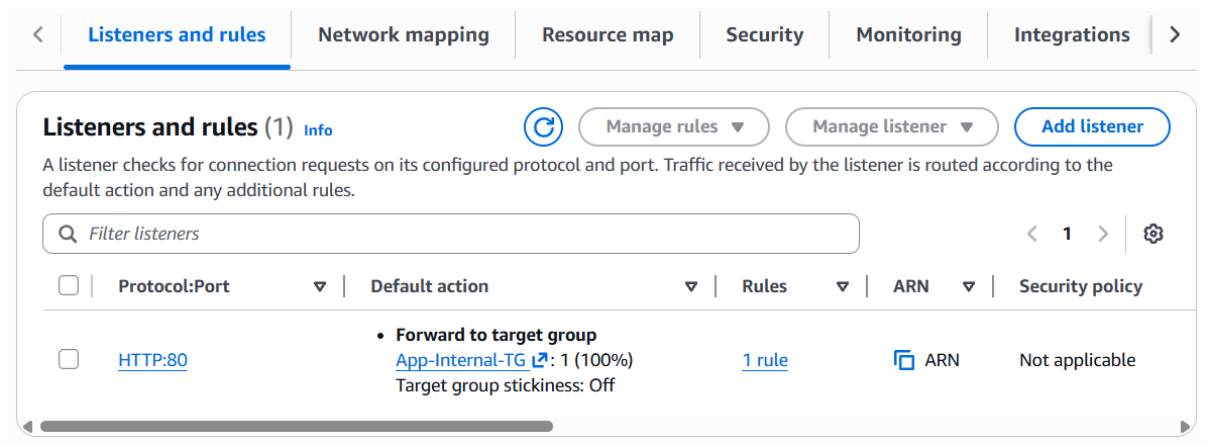
- Create the target group named **App-Interna-TG** to allow routes traffic from internal load balancer to application instance.



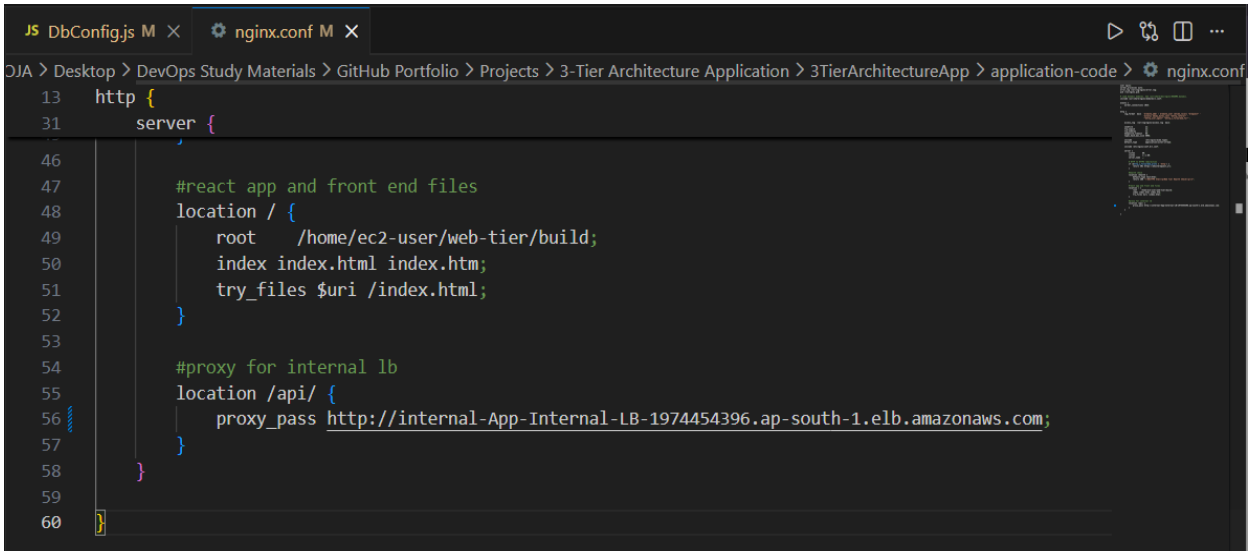


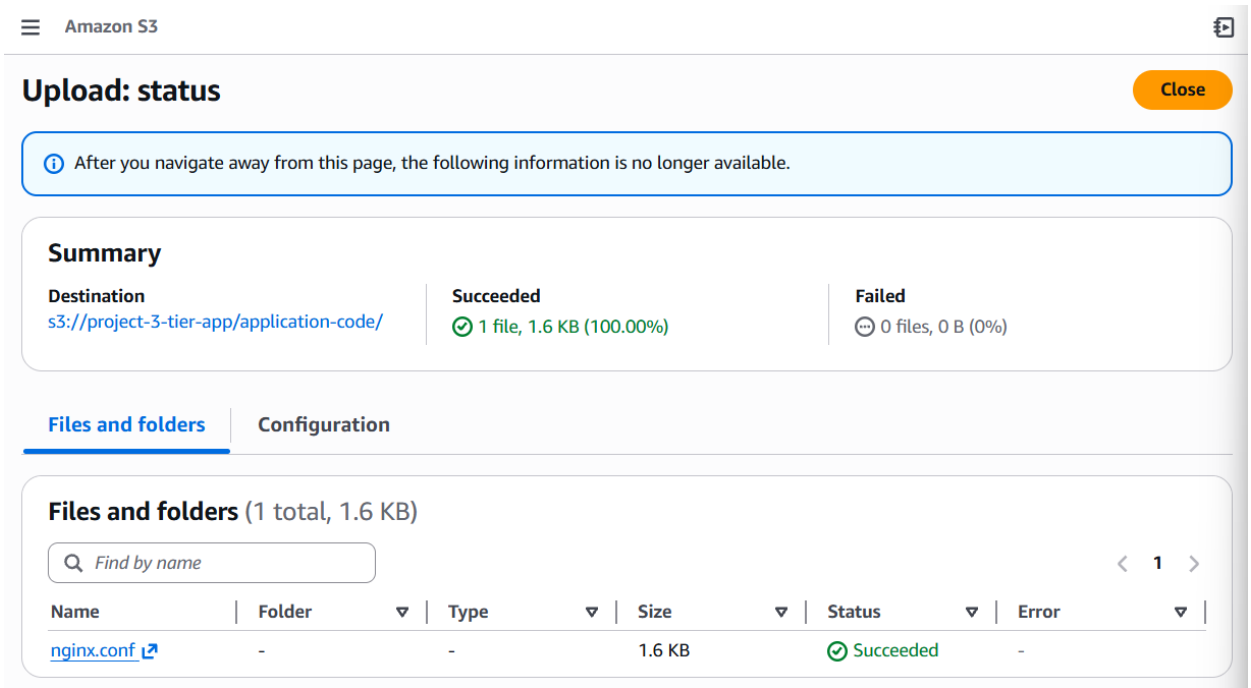
- Then create the load balancer named **App-internal-LB** configures the private subnets and select the App-Interna-TG target group.
- It allows secure communication between Web Tier and App Tier.





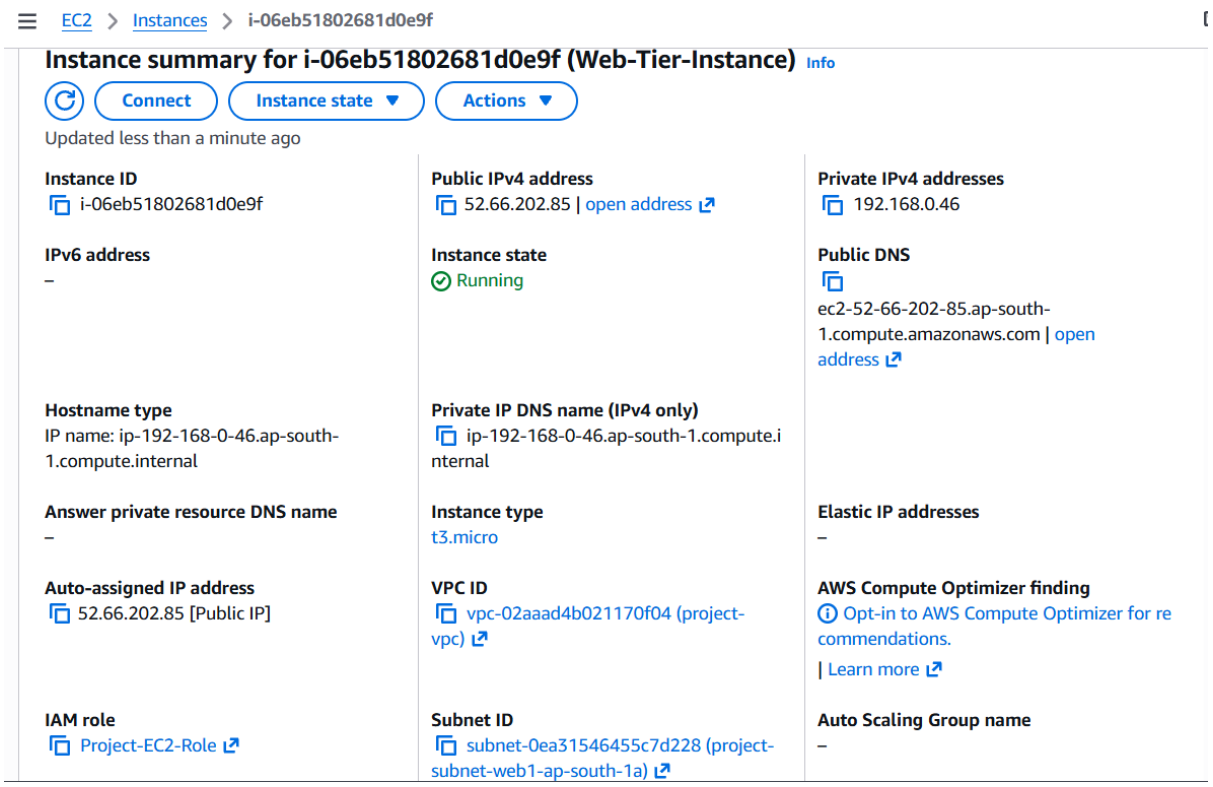
- Update the configuration with Internal Load Balancer DNS and update the file in s3 bucket. It ensures Web Tier connects to Application Tier securely.

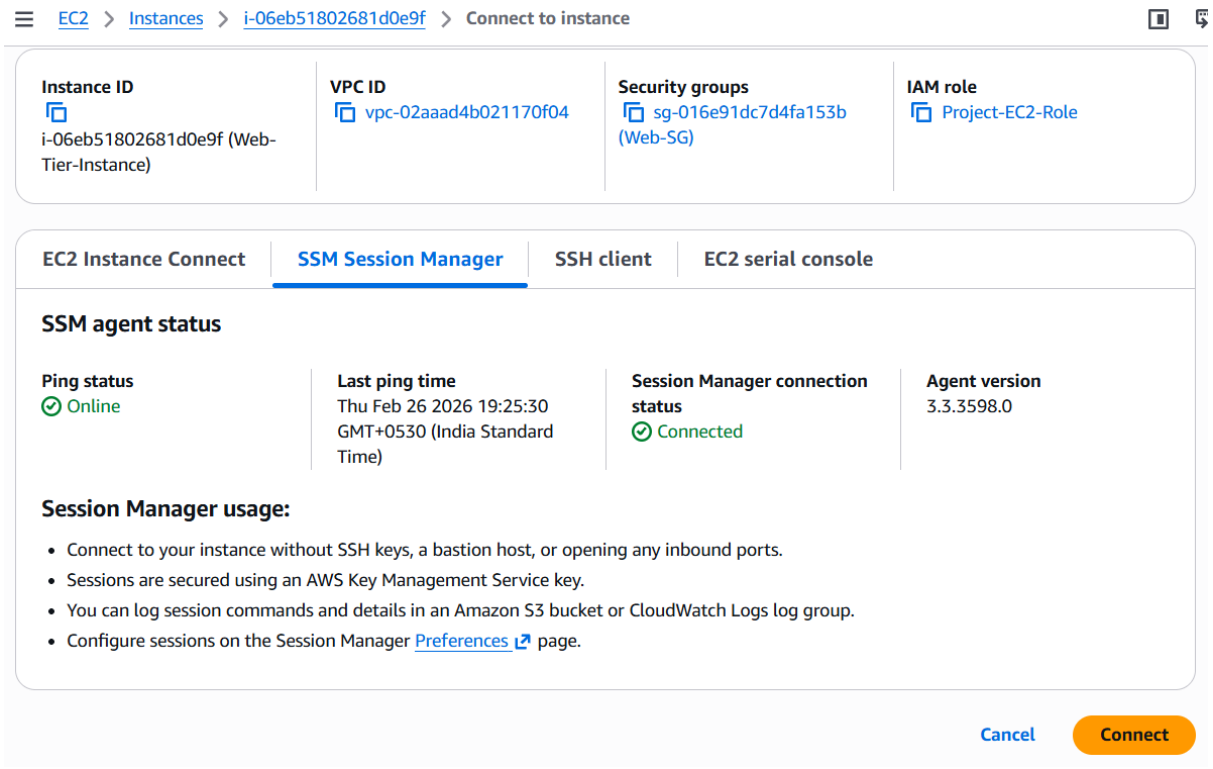




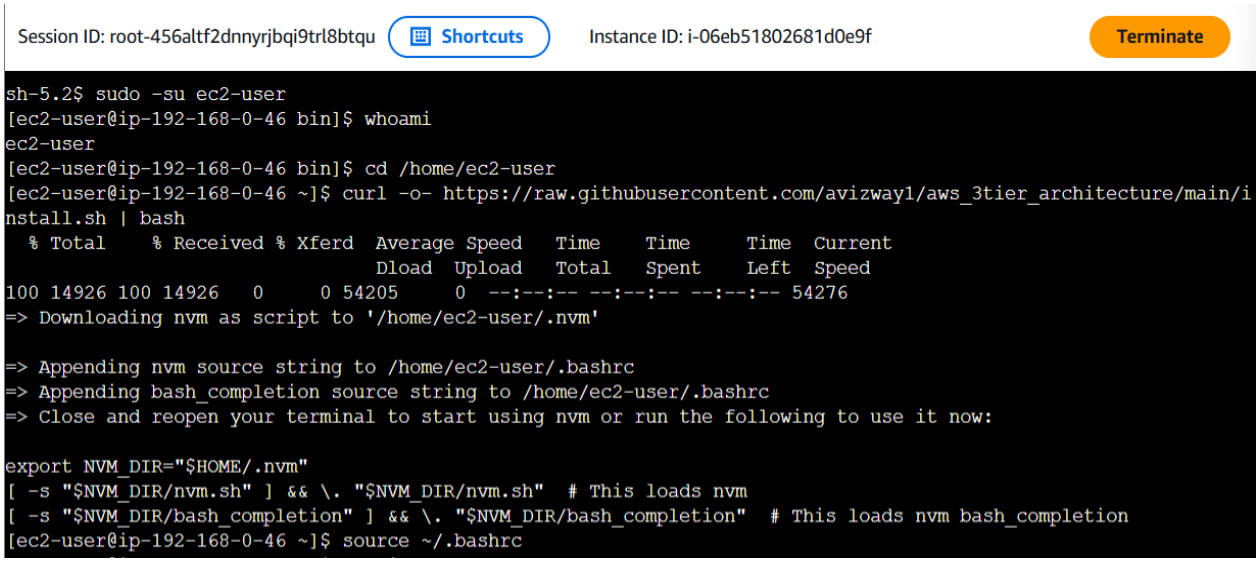
5. Web Tier Setup

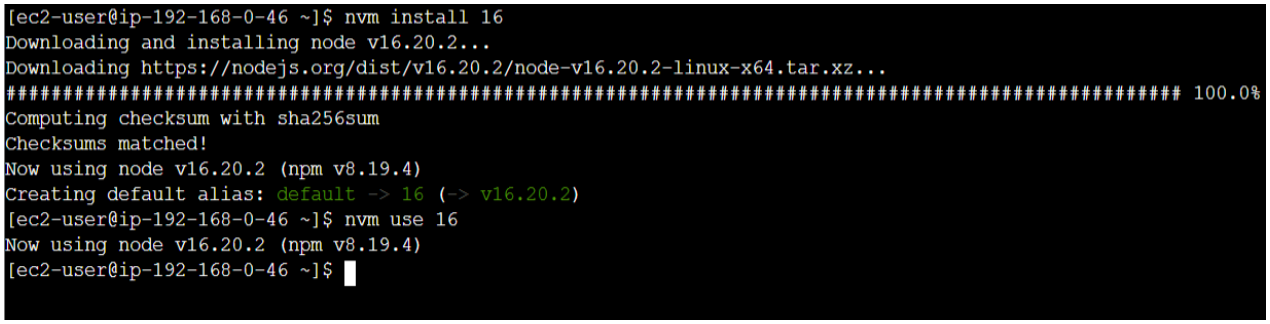
- Create web tier EC2 Instance **Web-Tier-Instance** in the public subnet and access it using SSM Session Management.



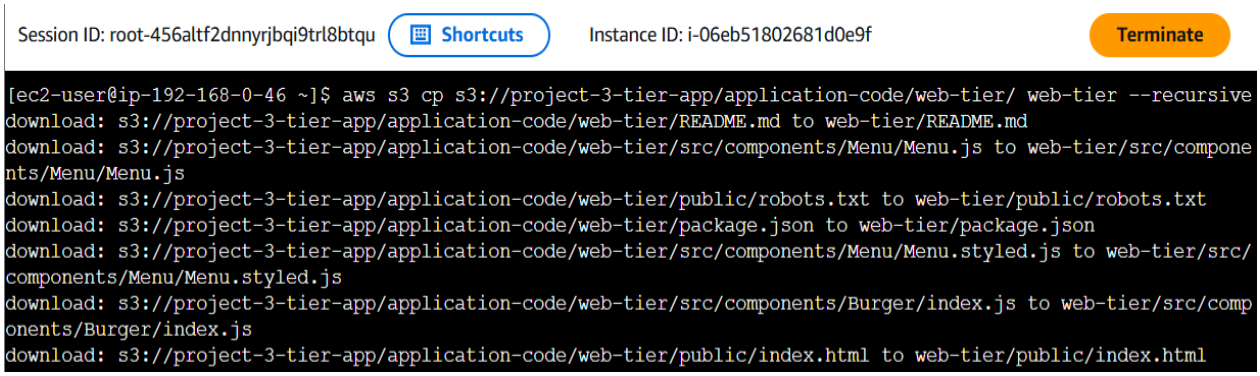


- Install Node.js and Nginx





- Download the web page related code from s3 and start the application.



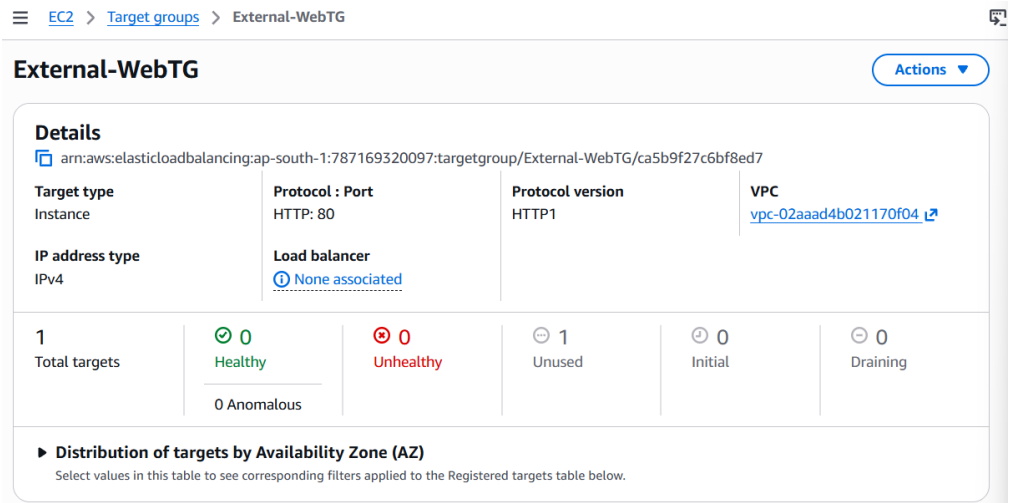
```
[ec2-user@ip-192-168-0-46 ~]$ ls
web-tier
[ec2-user@ip-192-168-0-46 ~]$ cd web-tier
[ec2-user@ip-192-168-0-46 web-tier]$ npm install
npm WARN EBADENGINE Unsupported engine {
npm WARN EBADENGINE   package: '@testing-library/dom@10.4.1',
npm WARN EBADENGINE   required: { node: '>=18' },
npm WARN EBADENGINE   current: { node: 'v16.20.2', npm: '8.19.4' }
npm WARN EBADENGINE }
```

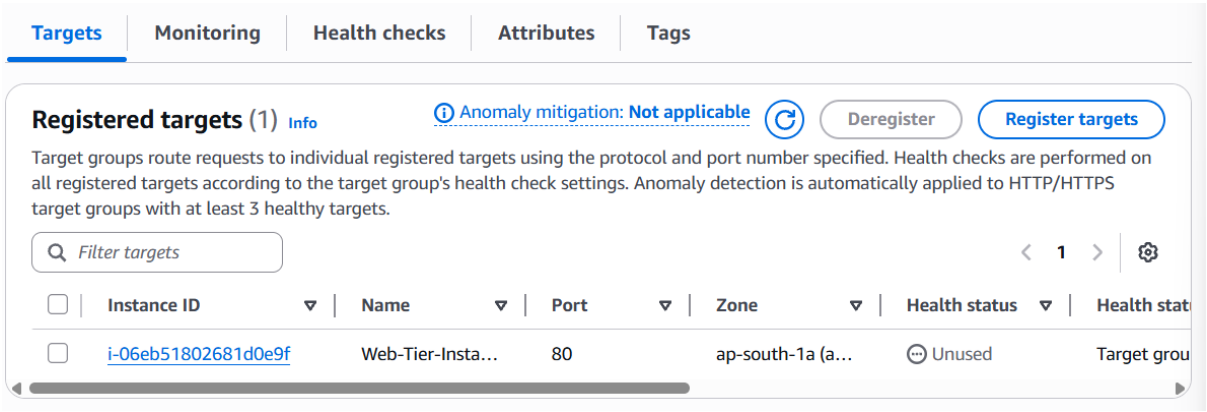
- Update nginx.conf file and set permissions so it allows Nginx to act as reverse proxy to application tier.

```
[ec2-user@ip-192-168-0-46 nginx]$ sudo dnf install nginx -y
Last metadata expiration check: 0:15:27 ago on Thu Feb 26 14:23:40 2026.
Package nginx-1:1.28.2-1.amzn2023.0.1.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-192-168-0-46 nginx]$ cd /etc/nginx
[ec2-user@ip-192-168-0-46 nginx]$ ls
conf.d                fastcgi_params          mime.types          scgi_params           win-utf
default.d             fastcgi_params.default  mime.types.default  scgi_params.default
fastcgi.conf          koi-utf                 nginx.conf          uwsgi_params
fastcgi.conf.default  koi-win                 nginx.conf.default  uwsgi_params.default
[ec2-user@ip-192-168-0-46 nginx]$ sudo rm nginx.conf
[ec2-user@ip-192-168-0-46 nginx]$ sudo aws s3 cp s3://project-3-tier-app/application-code/nginx.conf .
download: s3://project-3-tier-app/application-code/nginx.conf to ./nginx.conf
[ec2-user@ip-192-168-0-46 nginx]$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
[ec2-user@ip-192-168-0-46 nginx]$ sudo systemctl restart nginx
[ec2-user@ip-192-168-0-46 nginx]$
```

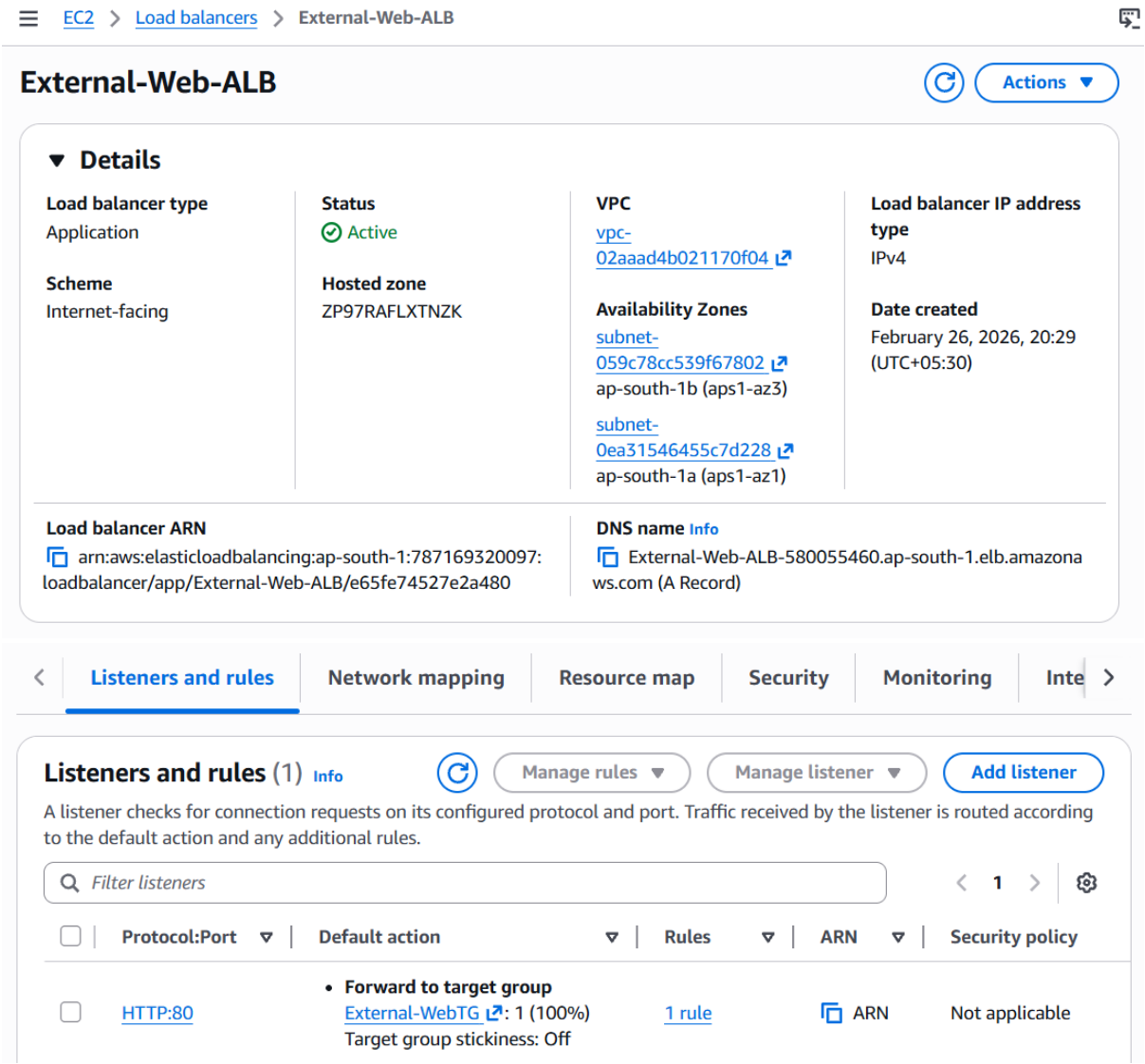
```
[ec2-user@ip-192-168-0-46 nginx]$ chmod -R 755 /home/ec2-user
[ec2-user@ip-192-168-0-46 nginx]$ sudo systemctl enable nginx
Created symlink /etc/systemd/system/multi-user.target.wants/nginx.service → /usr/lib/systemd/system/nginx.servi
ce.
[ec2-user@ip-192-168-0-46 nginx]$
```

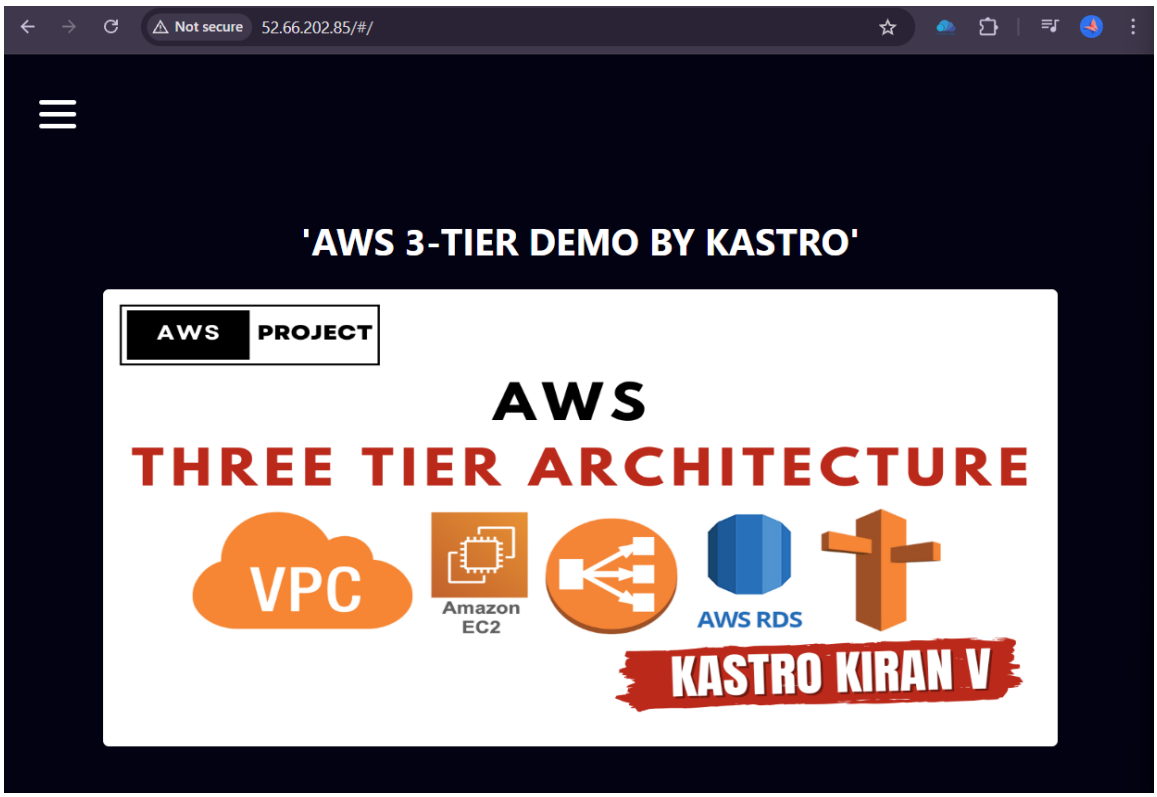
- Create the external target group named **External-WebTG** for Web-Tier-Instance

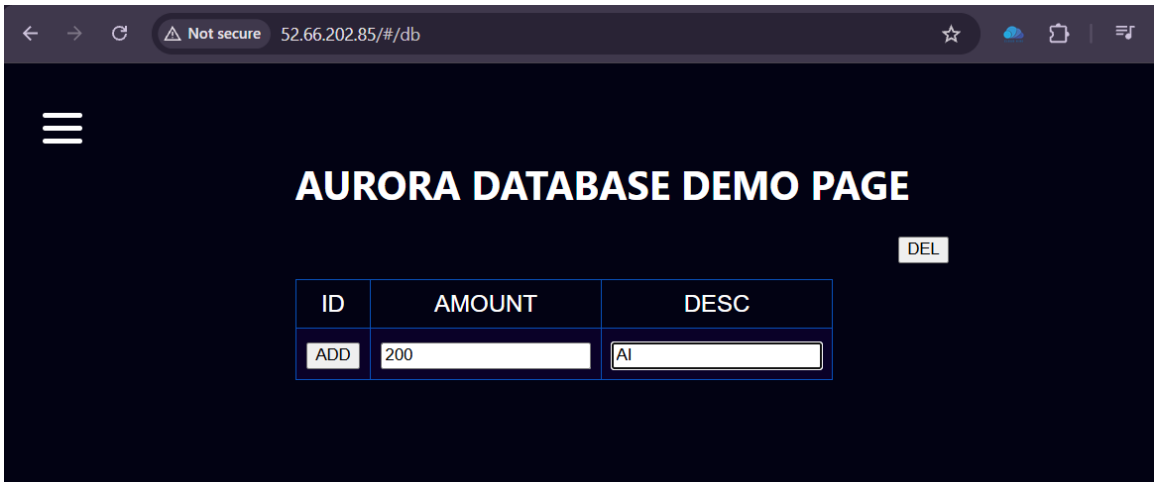




- Create the External Load Balancer named **External-Web-ALB** with External-WebTG in web tier subnets.
- We can access the Web page using the DNS name of the load balancer and also public ip of the web-tier-instance with port 80.







- This project demonstrates the successful deployment of a secure and highly available 3-Tier application architecture using AWS services such as VPC, EC2, RDS, S3, IAM and Load Balancers. By separating the web, application and database tiers into public and private subnets and using internal and external load balancers, the architecture ensures proper security, scalability and efficient communication between components. This implementation reflects practical experience in AWS infrastructure design, networking and deploying real-world production-style applications.